Universidad Fidélitas

Ingeniería en Sistemas de Computación

SC-403 – Desarrollo de Aplicaciones Web y Patrones

Práctica Programada #1

Profesor:

José Antonio Ortega González

Estudiantes:

Hernández Cordoba Isaac

Montero Hernández Josué David

Fecha de Entrega:

Martes, 24 de febrero

I Cuatrimestre, 2026

1. Objetivo del Proyecto

El objetivo fue desarrollar una aplicación web utilizando Spring Boot bajo el patrón MVC para administrar cursos y estudiantes.

El sistema permite realizar operaciones CRUD completas sobre ambas entidades, mantener persistencia en una base de datos MySQL, implementar fragmentos reutilizables en las vistas y soportar internacionalización en español e inglés.

2. Tecnologías Utilizadas

El proyecto fue desarrollado utilizando:

- Spring Boot como framework principal
- Spring MVC para el manejo de controladores y rutas
- Spring Data JPA / Hibernate para la persistencia
- MySQL como motor de base de datos
- Thymeleaf como motor de plantillas
- Bootstrap para el diseño visual
- Jakarta Validation para validaciones en las entidades

3. Arquitectura Implementada (MVC)

El sistema sigue el patrón Modelo–Vista–Controlador.

Modelo

Se definieron dos entidades JPA:

- Curso
- Estudiante

Ambas están mapeadas a tablas en MySQL y contienen validaciones para asegurar integridad de datos.

Vista

Las vistas fueron implementadas con Thymeleaf:

- index.html
- curso/listado.html

- curso/modificar.html
- estudiante/listado.html
- estudiante/modificar.html
- estudiante/porCurso.html

Las vistas renderizan información proveniente del modelo usando expresiones como:

th:text="#{clave}"
th:each="..."
th:field="..."

Controlador

Se implementaron:

- CursoController
- EstudianteController

Estos controladores reciben las solicitudes HTTP, interactúan con los servicios y devuelven las vistas correspondientes.

Flujo general

1. El usuario accede a una URL.
2. El controlador procesa la solicitud.
3. Se invoca la capa de servicio.
4. El servicio utiliza el repositorio para interactuar con la base de datos.
5. Se retorna una vista con datos en el modelo.
6. Thymeleaf genera el HTML final.

4. Ejecución del Sistema

La aplicación se ejecuta desde la clase:

PracticaProgramadaApplication

Spring Boot levanta un servidor embebido Tomcat automáticamente.

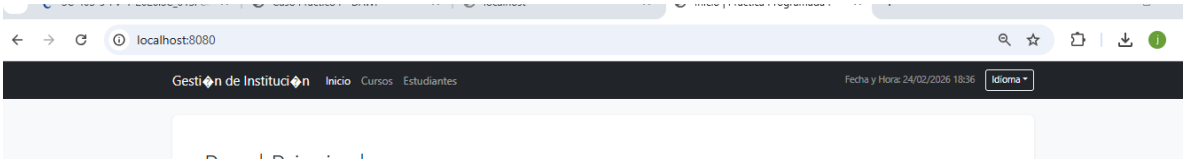
El sistema queda disponible en:

http://localhost:8080

Rutas principales:

- /
- /cursos/listado
- /estudiantes/listado

Captura:



5. Configuración de Base de Datos

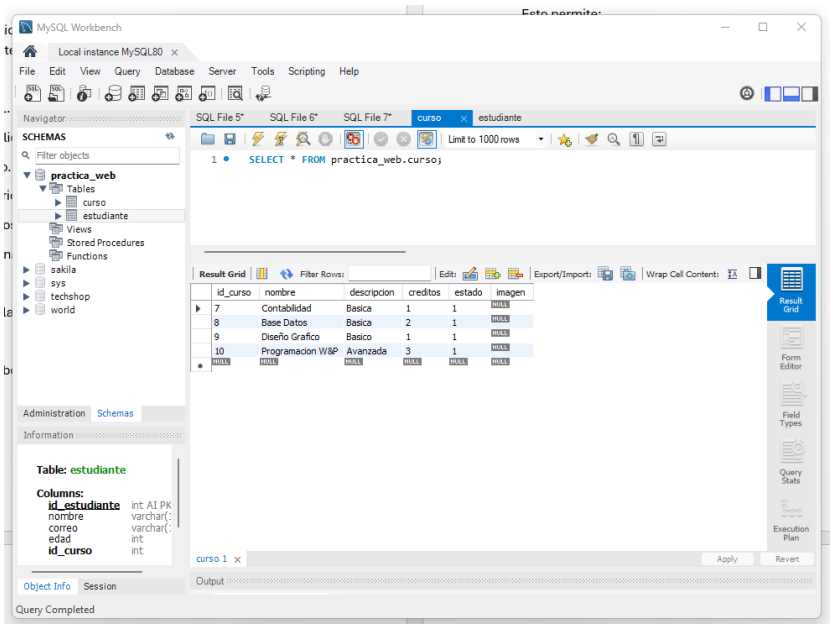
En el archivo application.properties se configuró la conexión:

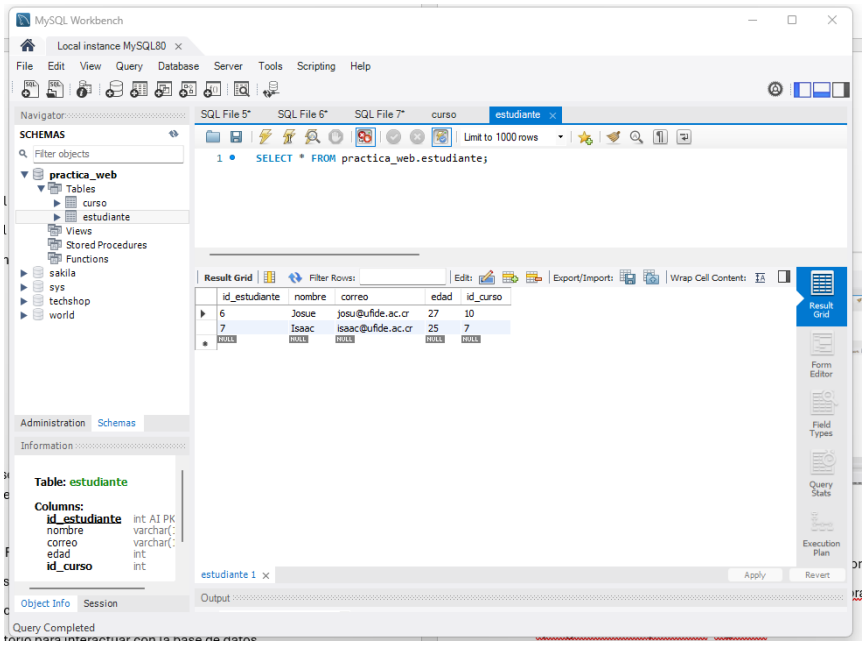
```
spring.datasource.url=jdbc:mysql://localhost:3307/practica_web
spring.datasource.username=usuario_web
spring.datasource.password=la_Clave
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Esto permite:

- Conectarse a MySQL
- Crear o actualizar tablas automáticamente
- Mostrar las consultas SQL en consola

Capturas:





6. Modelo de Datos

6.1 Entidad Curso

Tabla: curso

Campos principales:

- idCurso (clave primaria autoincremental)

- nombre
- descripcion
- creditos
- estado
- imagen

Se aplicaron validaciones como:

- @NotBlank
- @Size
- @Min
- @NotNull

Relación:

```
@OneToMany(mappedBy = "curso")
private List<Estudiante> estudiantes;
```

Esto indica que un curso puede tener múltiples estudiantes asociados.

6.2 Entidad Estudiante

Tabla: estudiante

Campos principales:

- idEstudiante (clave primaria autoincremental)
- nombre
- correo
- edad
- curso (clave foránea)

Relación:

```
@ManyToOne
@JoinColumn(name = "id_curso", nullable = false)
private Curso curso;
```

Esto establece que muchos estudiantes pertenecen a un mismo curso.

Capturas:

```java
package com.practica.domain;


import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import jakarta.validation.constraints.Min;
import java.util.List;

@Entity
@Table(name = "curso")
public class Curso {

    //Definicion de atributos de clase clasicos de JPA, de jo listo la auto incrementa
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_curso")
    private Integer idCurso;

    @NotBlank
    @Size(max = 100)
    private String nombre;

    @NotBlank
    @Size(max = 255)
    private String descripcion;

    @NotNull
    @Min(0)
    private Integer creditos;

    @NotNull
    private Boolean estado;

    @Size(max = 255)
    private String imagen;

    @OneToMany(mappedBy = "curso")
    private List<Estudiante> estudiantes;

    public Integer getIdCurso() { return idCurso; }
    public void setIdCurso(Integer idCurso) { this.idCurso = idCurso; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
```

```java
package com.practica.domain;

import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.validation.constraints.*;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "estudiante")
public class Estudiante {

    //Definicion de atributos de clase clasicos de JPA, de jo listo la auto incrementacion del id.
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_estudiante")
    private Integer idEstudiante;

    @NotBlank
    @Size(max = 100)
    private String nombre;

    @NotBlank
    @Email
    @Size(max = 100)
    private String correo;

    @NotNull
    @Min(1)
    private Integer edad;

    @ManyToOne
    @JoinColumn(name = "id_curso", nullable = false)
    private Curso curso;

    public Integer getIdEstudiante() { return idEstudiante; }
    public void setIdEstudiante(Integer idEstudiante) { this.idEstudiante = idEstudiante; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getCorreo() { return correo; }
```

7. Página Principal (Home)

La página principal se encuentra en:

http://localhost:8080/

Funciona como panel de navegación del sistema.

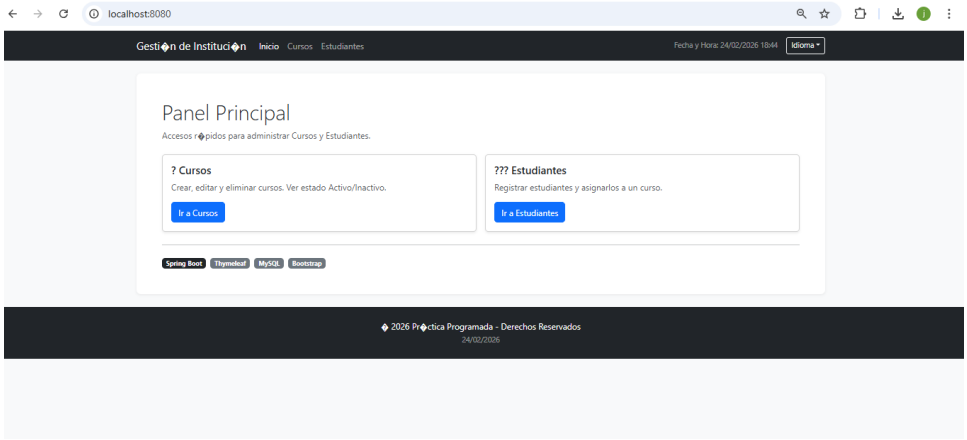
Desde esta vista se puede:

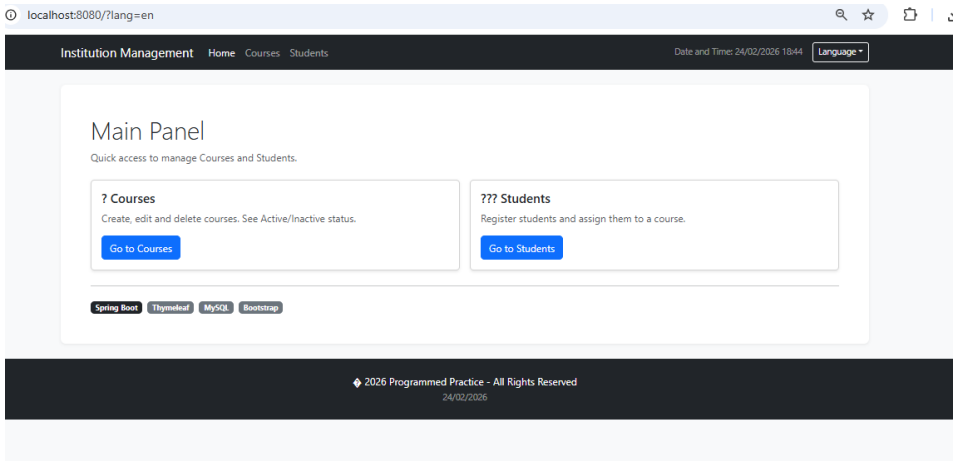
- Acceder al módulo de cursos
- Acceder al módulo de estudiantes
- Cambiar el idioma del sistema

Se utilizan fragmentos reutilizables mediante:

th:replace="fragments/header :: navbar('inicio')"

Capturas:





8. Módulo de Cursos

El controlador está definido con:

@RequestMapping("/cursos")

Funciones implementadas:

- Listado de cursos
- Guardar curso (con imagen opcional)
- Modificar curso
- Eliminar curso

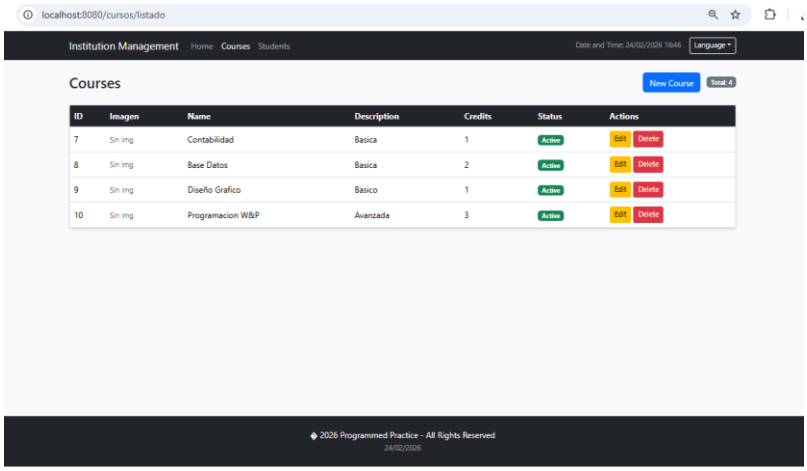
Endpoints:

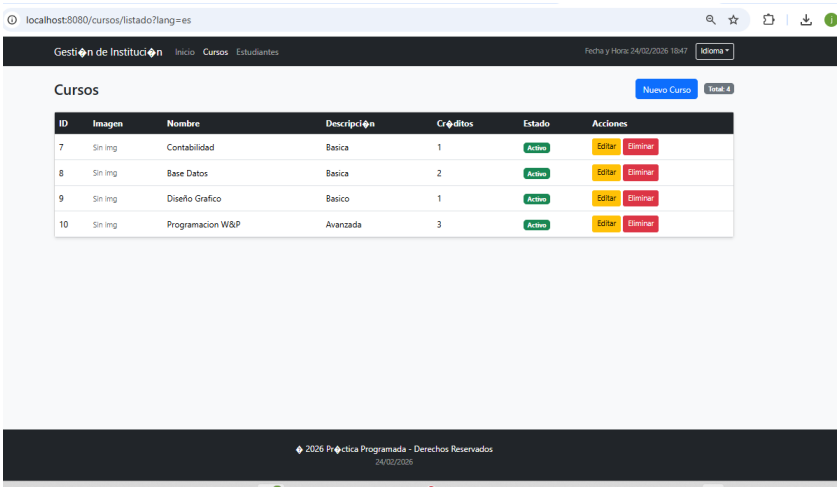
- GET /cursos/listado
- POST /cursos/guardar
- POST /cursos/eliminar
- GET /cursos/modificar/{idCurso}

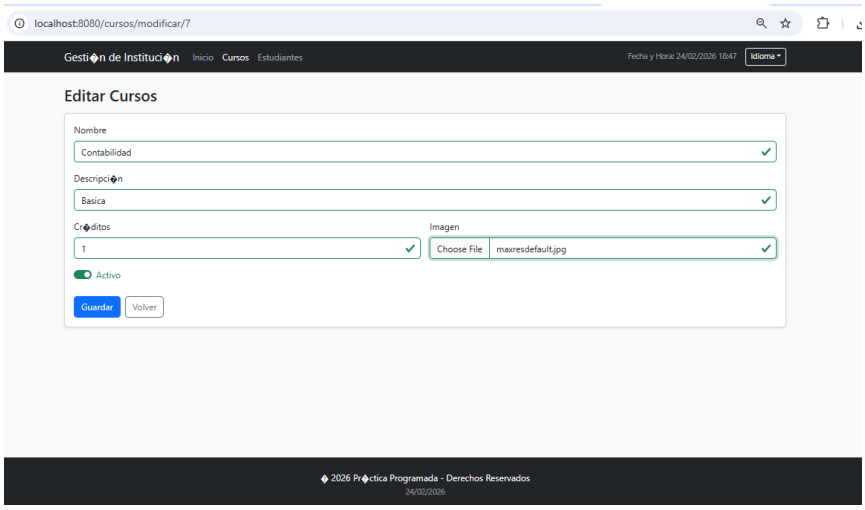
Incluye:

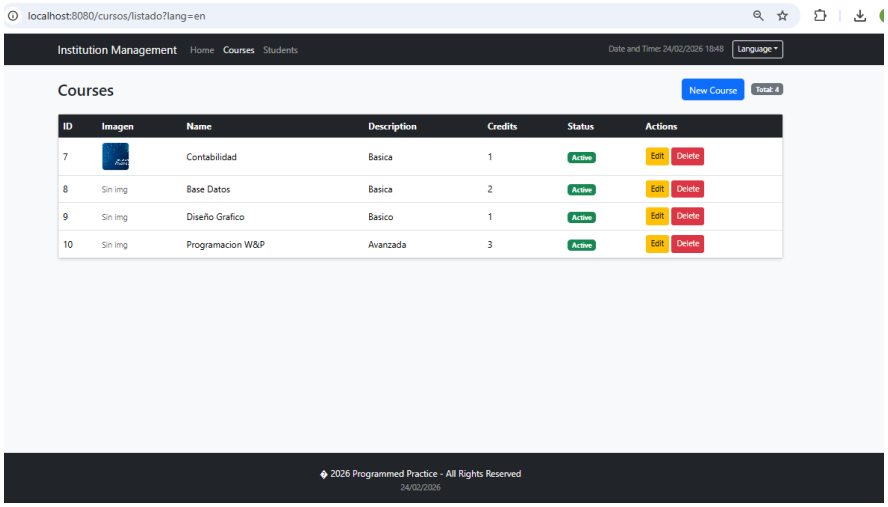
- Validación automática con @Valid
- Manejo de errores
- Uso de RedirectAttributes para mensajes
- Redirecciones después de operaciones

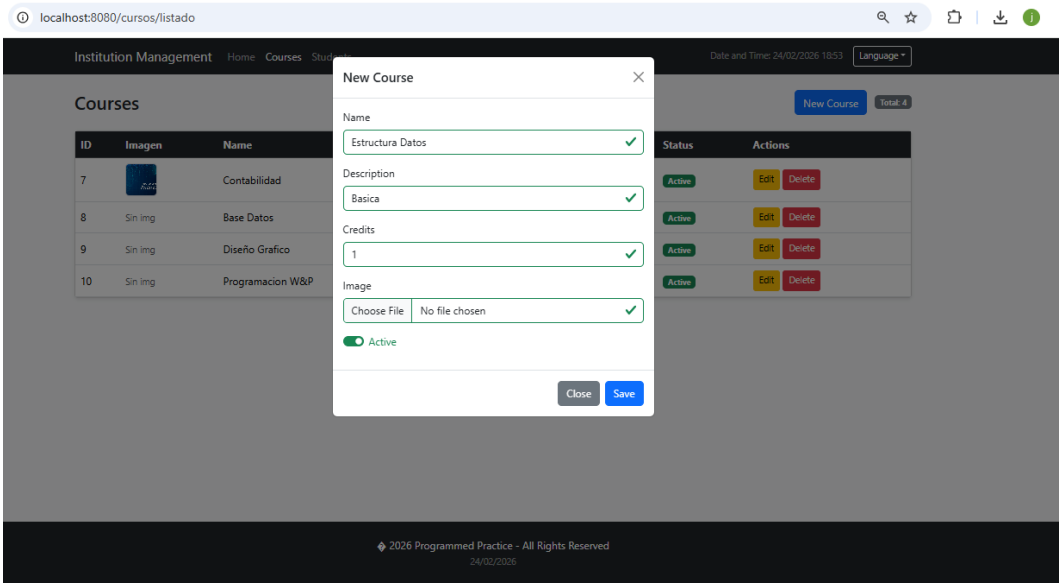
Capturas:

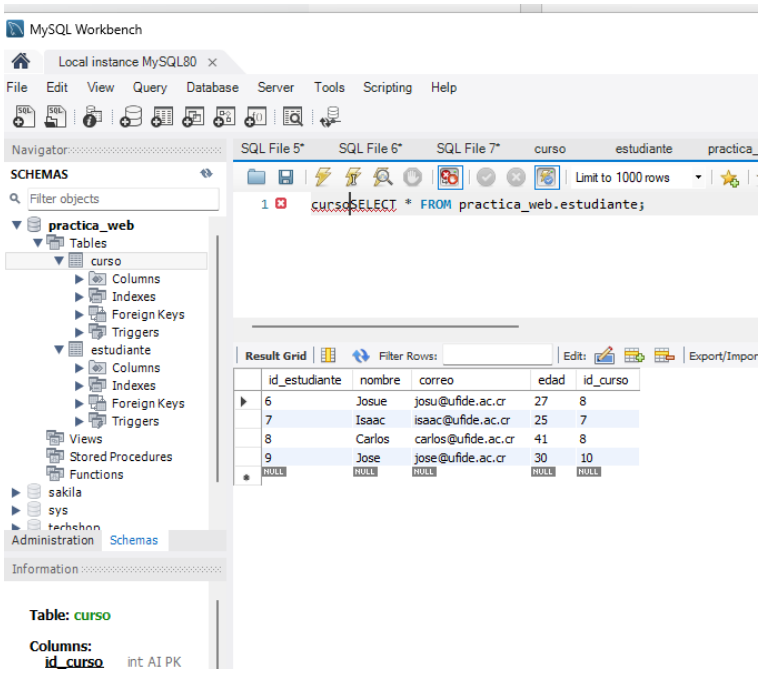



localhost:8080/cursos/listado?lang=es
Gesti◆n de Instituci◆n
Inicio
Cursos
Estudiantes
Fecha y Hora: 24/02/2026 18:47
Idioma
Cursos
Nuevo Curso
Total: 4
ID | Imagen | Nombre | Descripci◆n | Cr◆ditos | Estado | Acciones
7 | Sin img | Contabilidad | Basica | 1 | Activo | Editar Eliminar
8 | Sin img | Base Datos | Basica | 2 | Activo | Editar Eliminar
9 | Sin img | Diseño Grafico | Basico | 1 | Activo | Editar Eliminar
10 | Sin img | Programacion W&P | Avanzada | 3 | Activo | Editar Eliminar
◆ 2026 Pr◆ctica Programada - Derechos Reservados
24/02/2026


localhost:8080/cursos/modificar/7
Gesti◆n de Instituci◆n
Inicio
Cursos
Estudiantes
Fecha y Hora: 24/02/2026 18:47
Idioma
Editar Cursos
Nombre
Contabilidad
Descripci◆n
Basica
Cr◆ditos
1
Imagen
Choose File
maxresdefault.jpg
Activo
Guardar
Volver
◆ 2026 Pr◆ctica Programada - Derechos Reservados
24/02/2026


localhost:8080/cursos/listado?lang=en
Institution Management
Home
Courses
Students
Date and Time: 24/02/2026 18:48
Language
Courses
New Course
Total: 4
ID | Imagen | Name | Description | Credits | Status | Actions
7 | | Contabilidad | Basica | 1 | Active | Edit Delete
8 | Sin img | Base Datos | Basica | 2 | Active | Edit Delete
9 | Sin img | Diseño Grafico | Basico | 1 | Active | Edit Delete
10 | Sin img | Programacion W&P | Avanzada | 3 | Active | Edit Delete
◆ 2026 Programmed Practice - All Rights Reserved
24/02/2026





Nota: Se verificó completamente el funcionamiento del módulo CRUD (Crear, Editar y Eliminar), confirmando que las operaciones se reflejan correctamente en la base de datos MySQL.

9. Módulo de Estudiantes

Definido con:

@RequestMapping("/estudiantes")

Funciones:

- Listado de estudiantes
- Registro de estudiante asignado a curso
- Modificación
- Eliminación
- Filtrado por curso

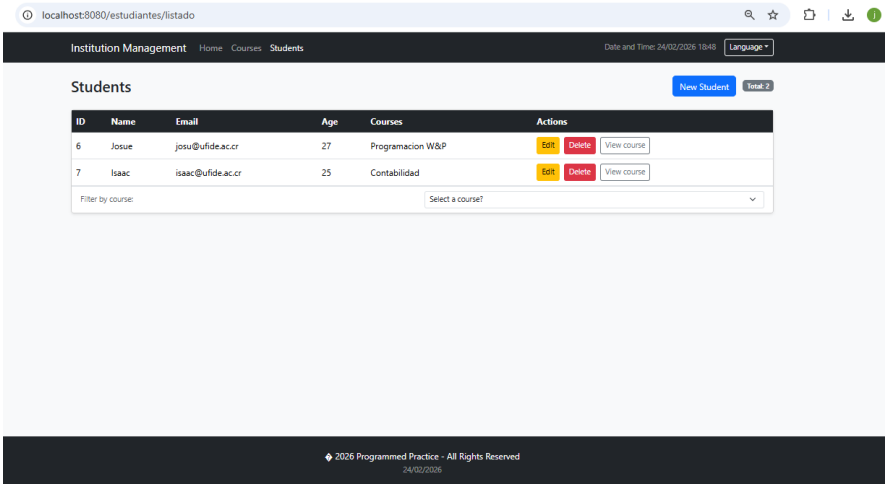
Endpoints:

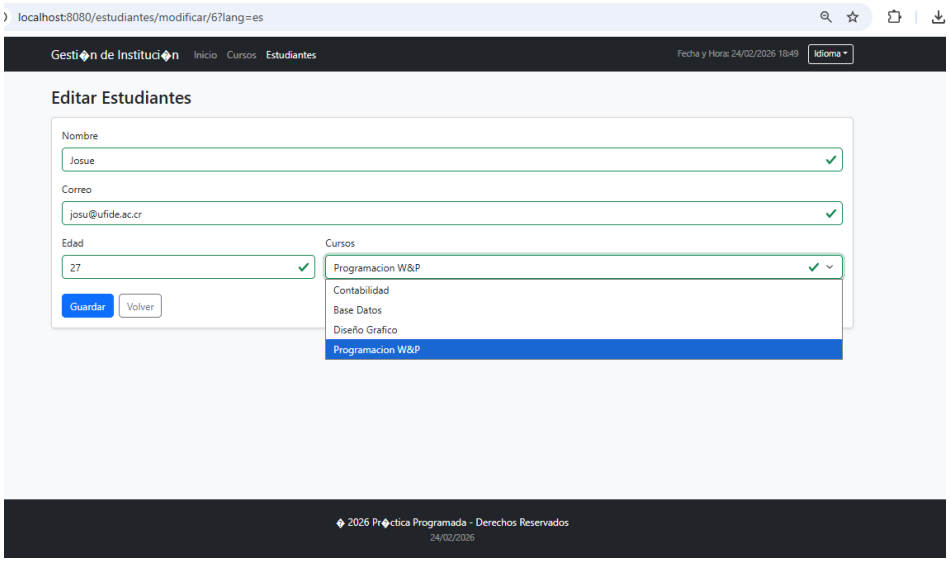
- GET /estudiantes/listado
- POST /estudiantes/guardar
- POST /estudiantes/eliminar
- GET /estudiantes/modificar/{idEstudiante}
- GET /estudiantes/por-curso/{idCurso}

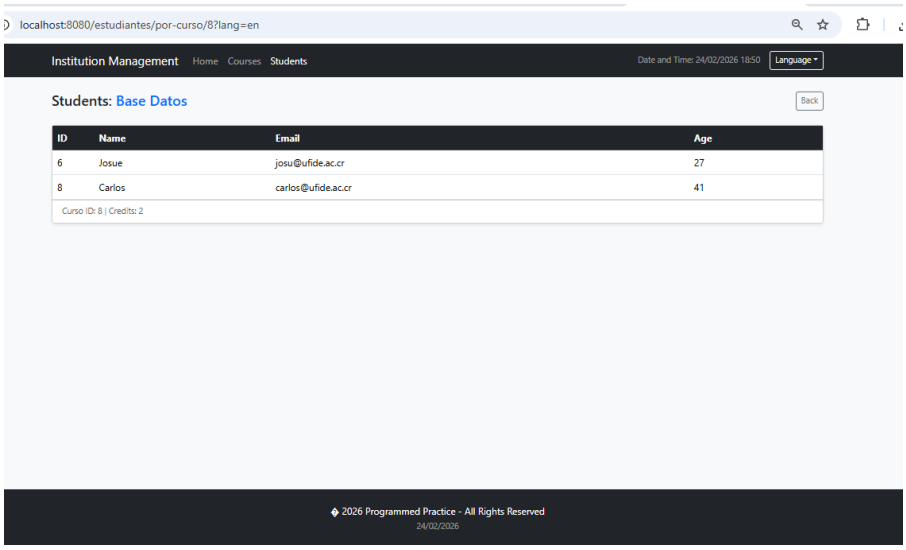
Se valida la existencia del curso antes de asignarlo.

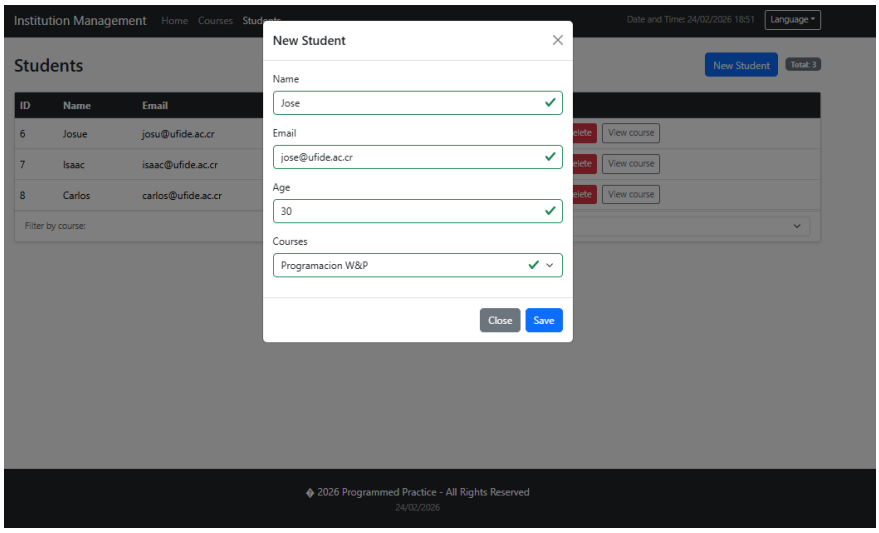
Se utilizan Optional para evitar errores de null.

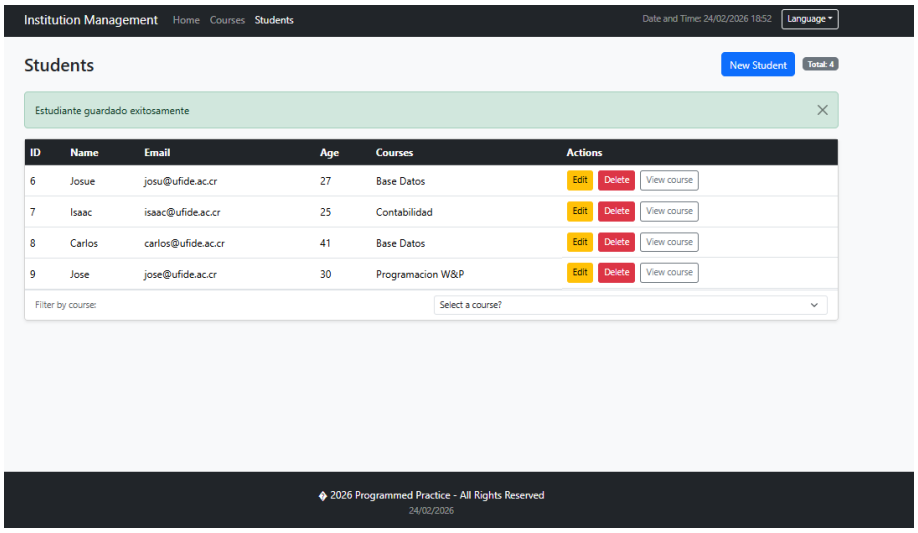
Capturas:

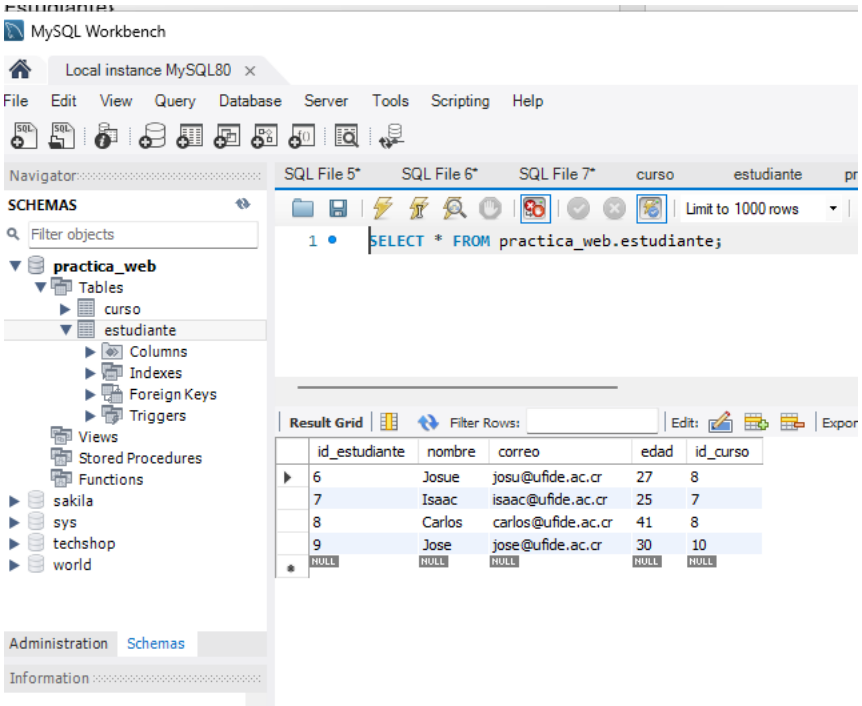



localhost:8080/estudiantes/modificar/6?lang=es
Gesti◆n de Instituci◆n
Inicio
Cursos
Estudiantes
Fecha y Hora: 24/02/2026 18:49
Idioma
Editar Estudiantes
Nombre
Josue
Correo
josu@ufide.ac.cr
Edad
27
Cursos
Programacion W&P
Contabilidad
Base Datos
Diseño Grafico
Programacion W&P
Guardar
Volver
◆ 2026 Pr◆ctica Programada - Derechos Reservados
24/02/2026


localhost:8080/estudiantes/por-curso/8?lang=en
Institution Management
Home
Courses
Students
Date and Time: 24/02/2026 18:50
Language
Students: Base Datos
Back
ID
Name
Email
Age
6
Josue
josu@ufide.ac.cr
27
8
Carlos
carlos@ufide.ac.cr
41
Curso ID: 8 | Credits: 2
◆ 2026 Programmed Practice - All Rights Reserved
24/02/2026


Institution Management
Home
Courses
Date and Time: 24/02/2026 18:51
Language
Students
New Student
Total: 3
ID
Name
Email
6
Josue
josu@ufide.ac.cr
7
Isaac
isaac@ufide.ac.cr
8
Carlos
carlos@ufide.ac.cr
View course
Filter by course:
New Student
Name
Jose
Email
jose@ufide.ac.cr
Age
30
Courses
Programacion W&P
Close
Save
◆ 2026 Programmed Practice - All Rights Reserved
24/02/2026





Nota: Se verificó el correcto funcionamiento del módulo CRUD y del filtrado de estudiantes por curso, confirmando que las operaciones realizadas en la interfaz web se reflejan adecuadamente en la base de datos y respetan la relación definida en el modelo de datos.

10. Fragmentos Reutilizables

Para evitar duplicación de código se implementaron:

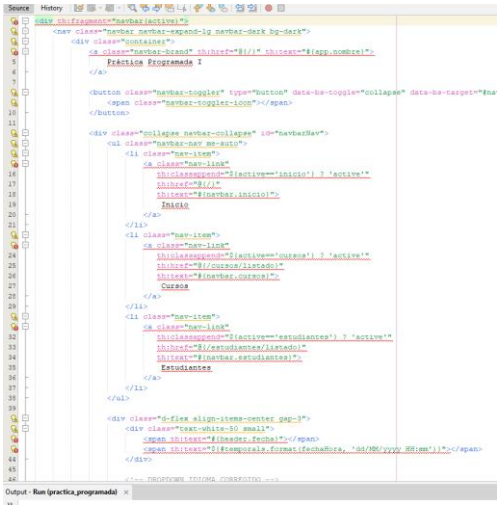
- header.html

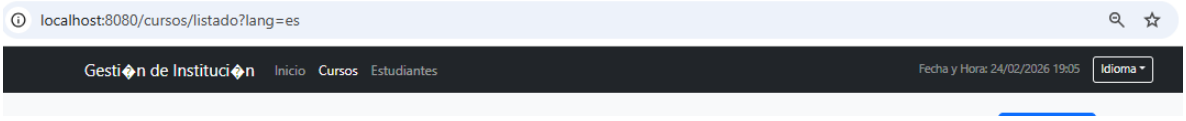
- footer.html

Uso en vistas:

th:replace="fragments/header :: navbar('cursos')"
th:replace="fragments/footer :: footer"

Esto permite mantener una estructura consistente en todas las páginas.





11. Internacionalización

El sistema soporta cambio de idioma mediante parámetro en la URL:

- ?lang=es
- ?lang=en

Ejemplo:

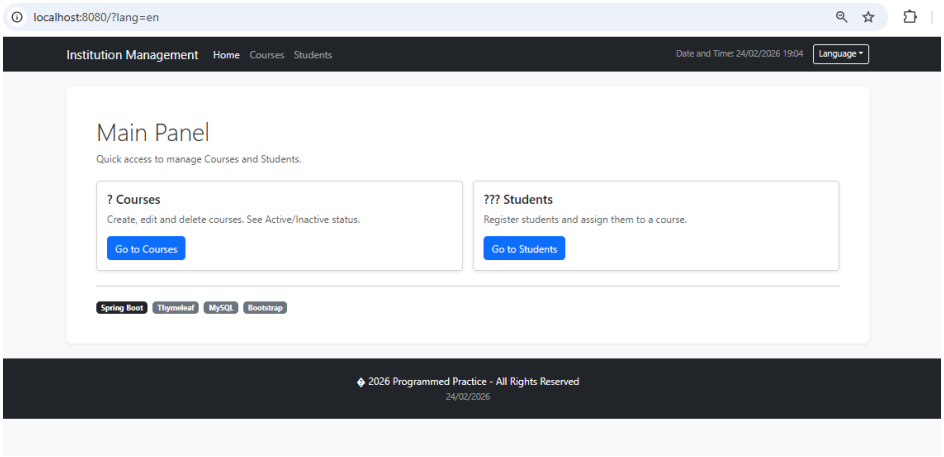
/cursos/listado?lang=en

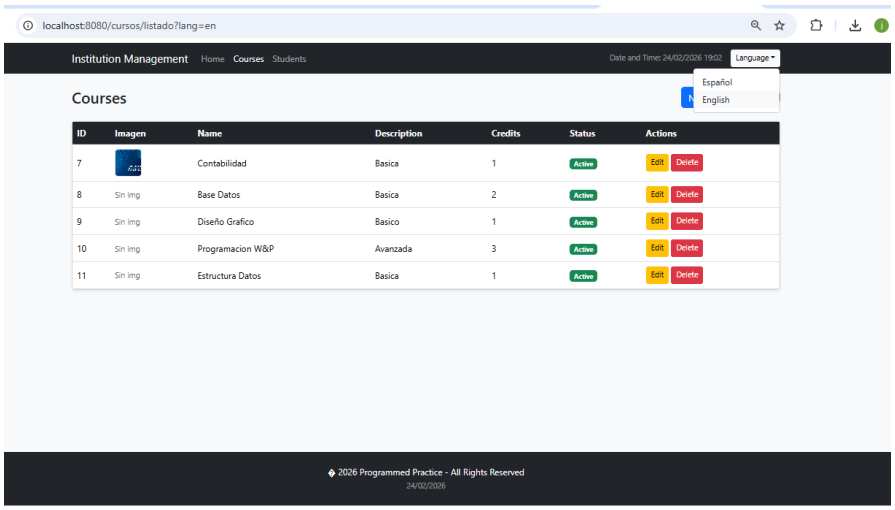
Se utilizan archivos:

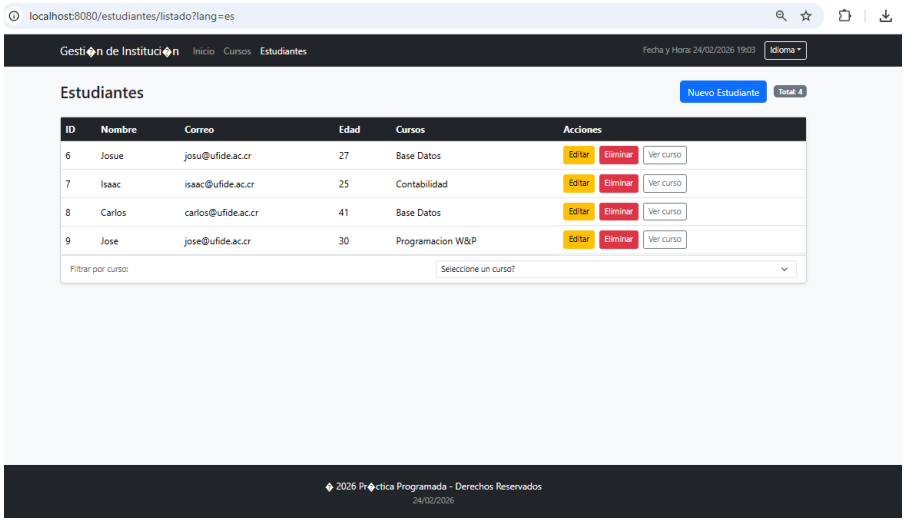
- messages.properties
- messages_en.properties

Los textos se renderizan con:

th:text="#{clave}"







12. Enlaces

Repositorio:
https://github.com/josu04Hdez/Practica_Programada_Uno.git

Video:
https://www.youtube.com/watch?v=YSnUCy50BB0

13. Conclusión

El proyecto implementa correctamente:

- CRUD completo para cursos y estudiantes
- Relación uno a muchos entre entidades
- Persistencia con JPA y MySQL
- Validaciones con Jakarta Validation
- Fragmentos reutilizables en Thymeleaf
- Internacionalización funcional
- Arquitectura MVC organizada

El sistema es funcional y cumple con los requisitos establecidos en la práctica.